

QUIC-Native Load Balancer

Proof of Concept

Fire-and-Forget Load Balancing via QUIC Server-Side Migration

PoC Documentation • July 2026

This document describes the design and implementation of a QUIC-native load balancer that leverages server-side connection migration (RFC 9000, Section 9.6) to achieve "fire-and-forget" load balancing. Unlike traditional load balancers (HAProxy, NGINX, Maglev) that remain in the data path for the entire connection lifetime, our approach handles only the initial handshake (1 RTT), selects a backend, and then steps out of the data path entirely. After migration, the client communicates directly with the backend server with zero frontend overhead.

1. Overview

Traditional load balancers sit in the data path for the **entire connection lifetime**. Every packet between client and server must traverse the LB, making it a throughput bottleneck and a single point of failure. This is true for both L4 (Maglev, LVS) and L7 (NGINX, Envoy, HAProxy) load balancers.

QUIC's server-side migration mechanism (the `preferred_address` transport parameter) offers a fundamentally different approach:

- The frontend handles **only the TLS handshake** (1 RTT), during which it selects a backend and advertises the backend's address as the preferred address.
- After the handshake, the client migrates directly to the backend. The frontend exports the cryptographic state and sends it to the backend.
- Once migration completes, the frontend has **zero per-connection state** and processes zero data packets for that connection.
- This is "fire-and-forget" load balancing: the frontend dispatches and the backend serves directly.

Key insight: The frontend's work is bounded by $O(1)$ per connection (handshake only), not $O(n)$ per packet. This eliminates the LB as a bandwidth bottleneck.

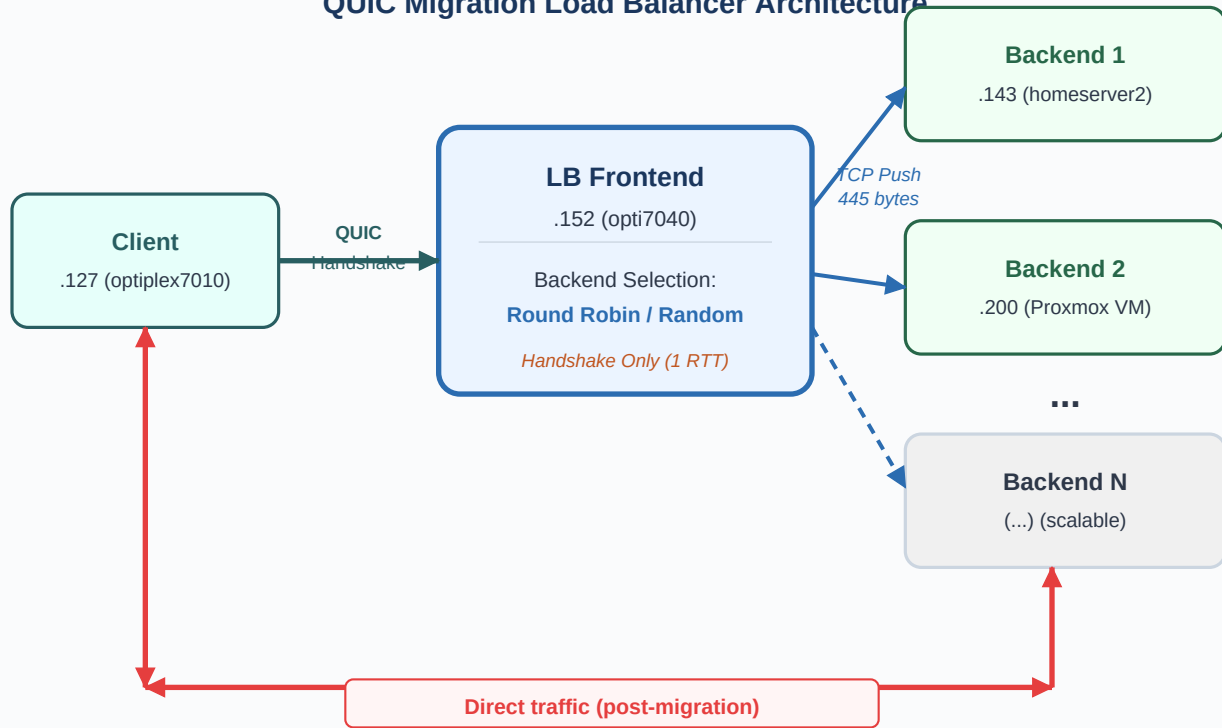
2. Architecture

The PoC runs on a four-machine testbed, all on the same LAN (141.217.168.x):

Role	Machine	IP Address	Binary
Frontend (LB)	opti7040	141.217.168.152	lb-frontend
Backend 1	homeserver2	141.217.168.143	preferred-server
Backend 2	Proxmox VM	141.217.168.200	preferred-server
Client	optiplex7010	141.217.168.127	Firefox / neqo-client

Architecture Diagram

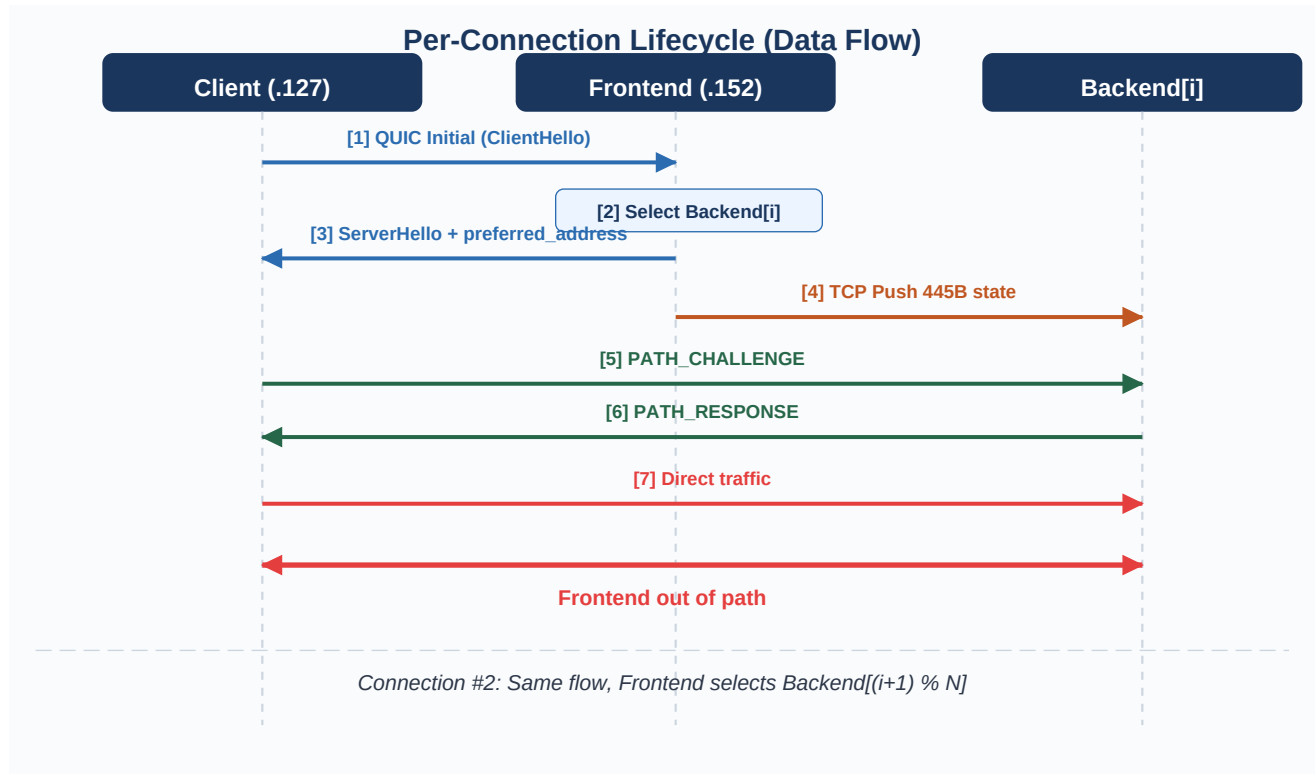
QUIC Migration Load Balancer Architecture



The diagram above shows the overall topology. The client initiates a QUIC handshake with the frontend. The frontend selects a backend via round-robin or random policy, advertises that backend as the preferred_address, and transfers cryptographic state. After path validation succeeds, the client communicates directly with the backend (shown in red). The frontend is completely out of the data path.

3. Per-Connection Data Flow

Each incoming connection follows a strict seven-step lifecycle. The frontend's involvement ends at step 4; after that, the backend handles all traffic directly.



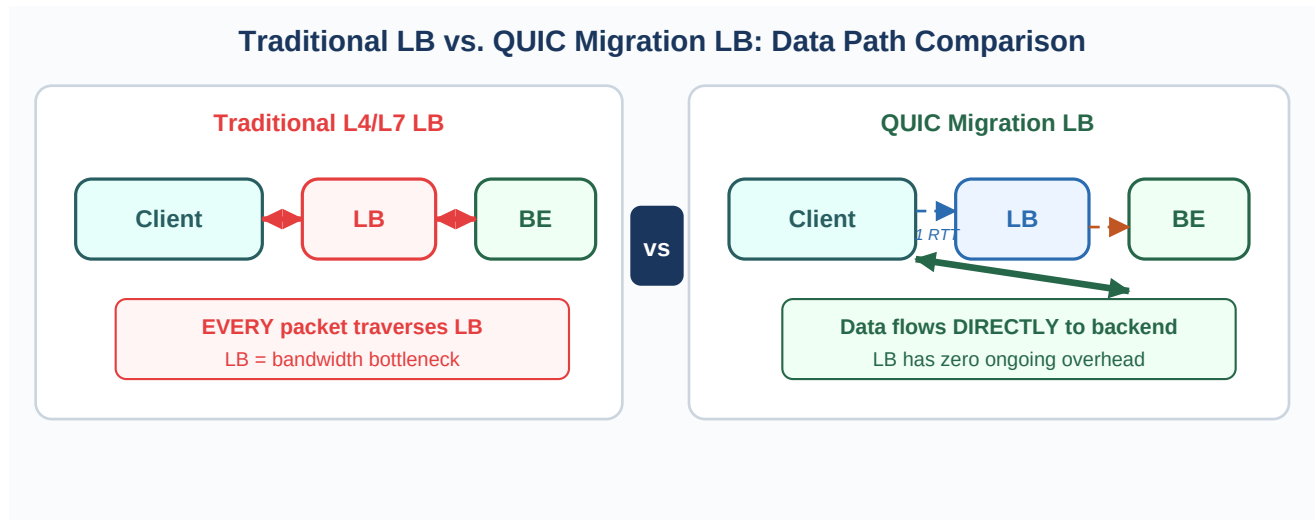
Connection Flow Detail

- 1. Client -> Frontend:** QUIC Initial packet containing the TLS ClientHello. This is the first contact; the client connects to the frontend's public address.
- 2. Frontend selects Backend[i]:** Based on the configured LB policy. For round-robin, this is $\text{counter} \% \text{num_backends}$; for random, a uniform random selection.
- 3. Frontend -> Client:** The TLS ServerHello includes the preferred_address transport parameter set to Backend[i]'s IP address and port. This completes the 1-RTT handshake.
- 4. Frontend -> Backend[i]:** The frontend exports ~445 bytes of migration state (TLS secrets, CIDs, packet numbers) and sends it to Backend[i] via TCP push to port 9999.
- 5. Client -> Backend[i]:** The client, having received the preferred address, sends a PATH_CHALLENGE to Backend[i] to validate the new path.
- 6. Backend[i] -> Client:** The backend imports the crypto state, decrypts the PATH_CHALLENGE, and responds with a PATH_RESPONSE. The client validates the response.
- 7. Client <-> Backend[i]:** Migration is complete. All subsequent HTTP/3 data flows directly between client and backend. The frontend has zero further involvement.

For Connection #2, the same flow repeats, but the frontend selects Backend[(i+1) % N] (round-robin) or a random backend. Each connection is independently load-balanced.

4. Traditional LB vs. QUIC Migration LB

The fundamental difference is where the load balancer sits in the data path:



The following table provides a detailed feature comparison across three LB approaches:

Feature	Maglev (L4)	NGINX/Envoy (L7)	QUIC Migration LB
LB in data path	Always	Always	Handshake only
TLS termination at LB	No (pass-through)	Yes	Handshake only
Sees plaintext	No	Yes	No (after migration)
Per-conn state at LB	Flow table	Session + buffers	None after migration
Bandwidth bottleneck	LB throughput	LB throughput	None (direct path)
Backend health check	Yes (active)	Yes (active)	Pre-migration only
Re-balancing mid-conn	Via flow reset	Via retry	Not possible
Protocol requirement	Any	HTTP/1.1, HTTP/2	QUIC (HTTP/3)

The QUIC migration LB combines the privacy benefits of L4 (no plaintext visibility) with the routing intelligence of L7 (backend selection at handshake time), while eliminating the persistent bandwidth bottleneck of both approaches.

5. Load Balancing Policies

The frontend supports multiple load balancing policies, selected via the LB_POLICY environment variable.

Round Robin (default)

Cycles through backends in order: [0, 1, 2, ..., N-1, 0, 1, ...]. This provides perfectly even distribution across backends and is fully deterministic.

Algorithm Detail

The round-robin algorithm maintains a single atomic counter that increments with each new connection. Backend selection is computed as:

```
selected_backend = counter % num_backends  
counter += 1
```

For example, with 3 backends and 9 connections, the assignment is: B0, B1, B2, B0, B1, B2, B0, B1, B2 -- exactly 3 connections each. This guarantees perfect distribution regardless of connection count (modulo the number of backends).

Random

Pseudo-random selection per connection. Each incoming connection is assigned to a backend chosen uniformly at random. Over many connections, this converges to even distribution but may have short-term imbalances. Selected with LB_POLICY=random.

Policy	Selection Logic	Distribution	State	Use Case
Round Robin	counter % N	Perfectly even	1 counter	Default; homogeneous backends
Random	rand() % N	Statistically even	None	Stateless; no coordination needed

Backend Statistics Tracking

Regardless of the selected policy, the frontend tracks per-backend statistics and prints a distribution table after each connection. This includes:

- **Connection count:** Total number of connections dispatched to each backend.
- **Percentage:** Proportion of total connections sent to each backend.
- **Last dispatch time:** Timestamp of the most recent connection to each backend.

Example output after 6 connections with round-robin and 2 backends:

```
Backend Distribution:  
141.217.168.143:4433 | 3 connections (50.0%)  
141.217.168.200:4433 | 3 connections (50.0%)
```

6. Implementation Details

Binary: lb-frontend

The load balancer frontend is implemented as the `lb-frontend` binary in the `migration-test` crate. It is a purpose-built binary separate from the standard `primary-server`.

Per-Connection Server Creation

The frontend creates a **new `Http3Server` instance for each incoming connection**. This is necessary because the `preferred_address` transport parameter is set during server construction and is included in the TLS handshake. Since each connection may be routed to a different backend, a fresh server instance with the correct preferred address must be created each time.

The lifecycle of each server instance is short-lived:

1. **Create:** New `Http3Server` with `Backend[i]`'s address as `preferred_address`.
2. **Handshake:** Process the QUIC handshake (1 RTT) with the client.
3. **Export:** Extract ~445 bytes of migration state (TLS secrets, CIDs, etc.).
4. **Transfer:** Send migration state to `Backend[i]` via TCP.
5. **Destroy:** Drop the server instance. No per-connection state persists.

A production implementation would ideally reuse a single server instance with dynamic per-connection `preferred_address` selection. The current per-connection instantiation is a PoC limitation that simplifies the integration with Neqo's API.

State Transfer

Migration state (~445 bytes) is sent via a direct TCP connection from the frontend to the selected backend's port 9999. This is a one-shot send with no response expected. The state includes:

Field	Size (bytes)	Description
TLS Application Secrets	~128	Read + Write AEAD keys for 1-RTT data
Connection IDs (CIDs)	~80	Client and server CIDs for packet routing
Client Address	~20	Client IP:port for response routing
QUIC Version	4	Protocol version negotiated during handshake
Packet Numbers	~16	Next expected RX/TX sequence numbers
Misc. Parameters	~197	Additional transport/crypto parameters

Per-Connection Stats

The frontend prints per-connection statistics including the selected backend, handshake duration, and state export time. It also maintains and prints a running backend distribution table showing how many connections have been sent to each backend.

7. Scalability Analysis

The key scalability advantage of the QUIC migration LB is that the frontend only handles the handshake phase of each connection. Once the handshake completes and state is transferred (~1 RTT), the frontend is free to serve new connections.

Frontend Cost Per Connection

Phase	Duration	CPU Cost	Bandwidth Cost
TLS Handshake	~1 RTT	ECDHE key exchange	~2-3 KB (hello + cert)
Backend Selection	O(1)	Counter increment or RNG	0
State Export	< 1 ms	Serialization	0 (internal)
State Transfer	< 1 ms (LAN)	TCP send	445 bytes
Connection Teardown	O(1)	Memory free	0
Total per conn	~1 RTT	~1 crypto op	~3 KB

Bandwidth Savings Calculation

Consider a scenario with 1,000 concurrent connections, each transferring 10 MB of data:

Metric	Traditional LB	QUIC Migration LB	Savings
Frontend bandwidth (handshake)	~3 MB	~3 MB	0%
Frontend bandwidth (data)	10 GB (all data)	0 B	100%
Total frontend bandwidth	~10 GB	~3 MB	99.97%
Frontend connections held	1,000 (ongoing)	0 (after migration)	100%
Frontend memory (per conn)	~50 KB x 1000 = 50 MB	0 (after migration)	100%

The frontend processes approximately 3 KB per connection (handshake only), then steps out. In a traditional LB, the same frontend must relay all 10 GB of data traffic. This represents a 99.97% reduction in frontend bandwidth utilization.

Connection Throughput

Since the frontend only handles handshakes, its connection throughput is limited by crypto operations (TLS 1.3 ECDHE), not bandwidth. A modern CPU can perform tens of thousands of ECDHE key exchanges per second per core, meaning a single-core frontend could theoretically handle 10,000+ new connections/second. In contrast, a traditional LB processing data for all connections is bandwidth-limited.

8. Environment Variables Reference

The following environment variables configure the load balancer frontend and backend binaries at runtime:

Variable	Binary	Default	Description
LB_POLICY	lb-frontend	roundrobin	Load balancing policy: roundrobin or random
STATE_TRANSFER	Both	tcp	State transfer backend: tcp, http, redis_kv, redis_ps, file
STATE_PORT	preferred-server	9999	TCP port for receiving migration state
REDIS_URL	Both	redis://141.217.168.200	Redis server URL (for redis_kv/redis_ps backends)
PKG_CONFIG_PATH	Build only	/nonexistent	Set to /nonexistent to use bundled NSS
PKG_CONFIG_LIBDIR	Build only	/nonexistent	Set to /nonexistent to use bundled NSS
LIBCLANG_PATH	Build only (homeserver2)	(unset)	Path to libclang on homeserver2: /usr/lib/llvm-14/lib
LD_LIBRARY_PATH	Runtime	(unset)	Set to NSS dist/Release/lib for runtime linking

9. How to Run

Start the components in the following order:

Step 1: Start Backend 1 (homeserver2, .143)

```
preferred-server 141.217.168.143:4433 9999
```

Step 2: Start Backend 2 (Proxmox VM, .200)

```
preferred-server 141.217.168.200:4433 9999
```

Step 3: Start Frontend (opti7040, .152)

```
lb-frontend 0.0.0.0:4433 141.217.168.143:4433 141.217.168.200:4433
```

The frontend binds to port 4433 and accepts two backend addresses as positional arguments. Additional backends can be appended to scale out.

Step 4: Connect from Client (.127)

Open Firefox and navigate to `https://141.217.168.152:4433/`. The first load triggers the TCP bootstrap (Alt-Svc). On refresh, Firefox uses HTTP/3 and the migration-based load balancing takes effect.

To select a policy, set the environment variable before starting the frontend:

```
LB_POLICY=random lb-frontend 0.0.0.0:4433 141.217.168.143:4433 141.217.168.200:4433
```

Using Shell Aliases

On the testbed machines, shell aliases are configured for convenience:

Machine	Alias	Description
homeserver2	preferred-up	Start backend (TCP push, immediate)
homeserver2	preferred-down	Stop backend (kill ports 9999/4433)
opti7040	bootstrap-up	Start TCP HTTPS Alt-Svc bootstrap
opti7040	bootstrap-down	Stop bootstrap server

10. Advantages Over Traditional Load Balancers

- **Zero steady-state bandwidth overhead:** After migration (~1 RTT), the frontend processes zero data packets. A 10 Gbps link serving 1,000 connections transfers zero bytes through the LB for ongoing data.
- **No single point of failure for data:** If the frontend goes down after migration, existing connections continue uninterrupted because they flow directly client-to-backend.
- **Privacy by design:** Unlike L7 reverse proxies, the frontend never sees plaintext application data. TLS is terminated only for the handshake; post-migration encryption keys are held solely by the backend.
- **Horizontal scaling of backends:** Adding a new backend requires only starting another preferred-server instance and adding its address to the frontend's backend list. No connection draining or LB reconfiguration is needed.
- **Reduced frontend hardware requirements:** Since the frontend only handles handshakes (CPU-bound, not bandwidth-bound), it can run on a much smaller machine than a traditional LB that must relay all traffic.

11. Limitations

One-shot migration: Once a connection migrates to a backend, it cannot be re-balanced. If the backend becomes overloaded after migration, the LB cannot intervene. This is inherent to the `preferred_address` mechanism.

Client support required: The client must support the `preferred_address` transport parameter. Firefox supports this; Chrome/Chromium does not (as of mid-2026). This limits real-world applicability until broader adoption.

Per-connection server instantiation: The current PoC creates a new `Http3Server` instance per connection. This is a PoC limitation, not a fundamental architectural constraint. A production implementation would reuse a single server instance with per-connection `preferred_address` selection.

No health checking: The PoC does not perform active backend health checks. If a backend is down, the frontend will still dispatch connections to it. A production system would need periodic health probes and backend removal on failure.

12. Test Plan

The following tests validate correctness, distribution, and performance of the load balancer PoC.

Test 1: Distribution Verification

Send 10 sequential connections to the frontend and verify even distribution across backends. With round-robin policy, expect exactly 5 connections per backend. With random policy, expect approximately even distribution (statistical test).

Test 2: Handshake + Migration Latency

Measure per-connection latency from initial ClientHello to completed migration (PATH_RESPONSE validated by client). Break down into: handshake time (frontend), state transfer time (TCP push), and path validation time (PATH_CHALLENGE/RESPONSE round trip to backend). Target: total migration latency under 2 RTT.

Test 3: Baseline Comparison

Compare with HAProxy in TCP mode (L4 pass-through) as the baseline. Metrics: connection setup latency, sustained throughput (the QUIC LB should show zero throughput overhead after migration), and frontend CPU utilization under load.

Test	Method	Success Criteria
Distribution	10 sequential connections	5/5 split (round robin); ~even (random)
Latency	Per-connection timing	Total migration < 2 RTT on LAN
Baseline	HAProxy TCP mode comparison	Zero throughput overhead after migration